

AKADEMIA TARNOWSKA

Katedra Informatyki | Informatyka II stopnia

KONSPEKT ZAJĘĆ LABORATORYJNYCH

Zaawansowany interfejs użytkownika

Laboratorium 4

React.js – komponenty i zarządzanie stanem

Parametr	Wartości
Przedmiot	Zaawansowany interfejs użytkownika (324796)
Forma zajęć	Laboratorium (LO) – ćwiczenia laboratoryjne
Numer zajęć	Laboratorium 4 z 14
Czas trwania	90 minut (1 jednostka dydaktyczna = 45 min)
Semestr / rok	Semestr 1 Rok akademicki 2025/2026
Prowadzący	mgr inż. Dominik Aksamit
Kontynuacja projektu	Aplikacja To-Do (projekt z Lab 2 i Lab 3 w Figma)
Technologia	React 18 + TypeScript 5 + Vite 5

01 Cele zajęć

Efekty uczenia się i oczekiwane kompetencje po ukończeniu laboratorium

1.1 Efekty uczenia się

Zajęcia realizują następujące kierunkowe efekty uczenia się:

Kod efektu	Treść efektu	Sposób weryfikacji
IN2_U01	Projektuje i implementuje interfejsy zgodnie z zasadami UCD	Obserwacja kodu + ocena aktywności
IN2_K03	Stosuje narzędzia do implementacji UI (React, TypeScript)	Ocena kodu źródłowego
IN2_K04	Samodzielnie rozwiązuje problemy projektowe w laboratorium	Obserwacja zachowań

1.2 Cele operacyjne

Po ukończeniu laboratorium student:

- tworzy funkcyjne komponenty React z propsami i własnym typowaniem w TypeScript (interfejsy Todo, FilterType),
- implementuje lokalne zarządzanie stanem za pomocą useState oraz rozumie kiedy zastąpić go useReducer,
- rozpoznaje problem props drilling i stosuje Context API jako rozwiązanie w wyizolowanym przykładzie dydaktycznym,
- realizuje kompletne operacje CRUD (dodawanie, edytowanie, usuwanie, przełączanie statusu) na liście zadań,
- implementuje filtrowanie listy (wszystkie / aktywne / ukończone) oraz licznik zadań w nagłówku aplikacji.

1.3 Powiązanie z projektem ciągłym

Laboratorium 4 stanowi pierwszy etap implementacyjny projektu semestralnego. Studenci przenoszą projekt hi-fi z Figma (opracowany na Lab 2–3) do środowiska React, realizując rdzeń biznesowy aplikacji To-Do. Kolejne laboratoria (5–14) będą rozszerzać tę bazę o biblioteki komponentów, responsywność, API REST i animacje.

02 Brief projektowy – aplikacja To-Do

Specyfikacja funkcjonalna zadania do implementacji

2.1 Kontekst i cel zadania

Zadaniem studentów jest zaimplementowanie w React 18 z TypeScript listy zadań (To-Do App), której projekt wizualny został opracowany w Figma na Lab 2 i Lab 3. Implementacja powinna wiernie odwzorowywać hi-fi prototype – hierarchię komponentów, kolorystykę i rozmieszczenie elementów.

2.2 Wymagania funkcjonalne

Operacje CRUD

- Dodawanie nowego zadania przez formularz (pole tekstowe + przycisk lub Enter)
- Edycja tytułu istniejącego zadania (tryb edycji w miejscu lub modal)
- Usuwanie zadania z listy (przycisk usuń przy każdym elemencie)
- Przełączanie statusu zadania: aktywne ↔ ukończone (checkbox)

Filtrowanie

- Pasek filtrowania z trzema trybami: Wszystkie | Aktywne | Ukończone
- Aktywny filtr wyróżnia się wizualnie (styl active)
- Lista aktualizuje się natychmiast po zmianie filtru (bez przeładowania)

Licznik zadań

- Nagłówek lub pasek boczny wyświetla: liczbę zadań aktywnych / łączną liczbę zadań
- Licznik aktualizuje się reaktywnie przy każdej zmianie stanu listy

2.3 Wymagania techniczne

- React 18 z hookami funkcyjnymi (bez klas)
- TypeScript 5 w trybie strict – wszystkie typy muszą być zdefiniowane
- Vite 5 jako bundler i dev server
- Brak zewnętrznych bibliotek komponentów UI na tym etapie (czysty CSS lub inline styles)
- Struktura komponentów zgodna z instrukcją krok po kroku (patrz Sekcja 4)

03 Starter kit

Konfiguracja projektu

3.1 Struktura katalogów

Po rozpakowaniu starter kitu student otrzymuje następującą strukturę:

```
todo-app/
├── src/
│   ├── components/
│   │   ├── AddTodoForm.tsx
│   │   ├── FilterBar.tsx
│   │   ├── TodoItem.tsx
│   │   └── TodoList.tsx
│   ├── context/
│   │   └── TodoContext.tsx          // Sekcja 4, Krok 5
│   ├── hooks/
│   │   └── useLocalStorage.ts       // Rozszerzenie zaawansowane
│   ├── reducers/
│   │   └── todoReducer.ts          // Sekcja 4, Krok 4
│   ├── types/
│   │   └── todo.types.ts
│   ├── App.tsx
│   └── main.tsx
├── index.html
├── package.json
├── tsconfig.json
└── vite.config.ts
```

3.2 package.json

```
{
  "name": "todo-app-lab4",
  "version": "1.0.0",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "lint": "eslint src --ext ts,tsx"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "framer-motion": "^11.0.0"
  }
}
```

```
  },
  "devDependencies": {
    "@types/react":      "^18.3.1",
    "@types/react-dom":  "^18.3.1",
    "@vitejs/plugin-react": "^4.3.1",
    "typescript":        "^5.5.3",
    "vite":              "^5.4.2"
  }
}
```

3.3 tsconfig.json (tryb strict)

```
{
  "compilerOptions": {
    "target":      "ES2020",
    "module":      "ESNext",
    "lib":          ["ES2020", "DOM", "DOM.Iterable"],
    "moduleResolution": "bundler",
    "jsx":          "react-jsx",
    "strict":       true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src"]
}
```

3.4 vite.config.ts

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: { port: 3000, open: true }
});
```

3.5 Typy bazowe – src/types/todo.types.ts

```
// Typ reprezentujący jedno zadanie
export interface Todo {
  id:      string;
  title:   string;
  completed: boolean;
```

```
    createdAt: Date;
  }

  // Możliwe wartości filtra
  export type FilterType = 'all' | 'active' | 'completed';

  // Typy akcji dla useReducer (Krok 4)
  export type TodoAction =
    | { type: 'ADD';    payload: string }
    | { type: 'TOGGLE'; payload: string }
    | { type: 'DELETE'; payload: string }
    | { type: 'EDIT';   payload: { id: string; title: string } };

```

3.6 Uruchomienie projektu

Polecenia do wykonania przed zajęciami lub na początku laboratorium:

```
# Sklonuj repozytorium lub rozpakuj archiwum
cd todo-app-lab4

# Zainstaluj zależności
npm install

# Uruchom serwer deweloperski
npm run dev

# Aplikacja dostępna pod: http://localhost:3000

```

04 Instrukcja krok po kroku

5 kroków × ok. 15 minut = 75 minut aktywnej implementacji

KROK 1

Typy TypeScript + komponent TodoItem | ~15 min

Cel kroku

Zapoznanie z typami zdefiniowanymi w starter kicie oraz implementacja najprostszego komponentu listy.

Zadanie

Plik `src/components/TodoItem.tsx` zawiera gotowy szkielet. Uzupełnij oznaczone miejsca.

Komponent `TodoItem` renderuje pojedyncze zadanie z:

- checkboxem przełączającym stan `completed`,
- tytułem zadania (przekreślonym gdy `completed === true`),
- przyciskiem usuwania.

Kluczowe pojęcia

- Props i ich typowanie: `interface TodoItemProps { todo: Todo; onToggle: ...; onDelete: ... }`
- Warunkowe stylowanie: `style={{ textDecoration: todo.completed ? 'line-through' : 'none' }}`
- Unikalny klucz `key` przy renderowaniu listy – dlaczego nie należy używać indeksu tablicy

KROK 2

AddTodoForm + `useState` dla formularza | ~15 min

Cel kroku

Implementacja formularza dodawania zadań z lokalną kontrolą stanu przez `useState`.

Zadanie

W pliku `src/components/AddTodoForm.tsx` uzupełnij brakujące linie:

- stan `inputValue`: `string` do śledzenia wartości pola tekstowego,
- handler `handleSubmit` tworzący nowe `Todo` (id generuj przez `crypto.randomUUID()`),
- walidacja: blokowanie dodania pustego tytułu (`trim()`).

Fragment do analizy – schemat komponentu

```
interface AddTodoFormProps {
  onAdd: (title: string) => void;
}

export function AddTodoForm({ onAdd }: AddTodoFormProps) {
  // TODO (1): zadeklaruj stan inputValue za pomocą useState<string>
```

```
// const [inputValue, setInputValue] = ...

const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  // TODO (2): sprawdź, czy inputValue.trim() nie jest pusty
  //           jeśli tak: wywołaj onAdd(inputValue.trim())
  //           i zresetuj inputValue do pustego stringa
};

return (
  <form onSubmit={handleSubmit}>
    <input value={inputValue} onChange={e => setInputValue(e.target.value)} />
    <button type='submit'>Dodaj</button>
  </form>
);
}
```

KROK 3

TodoList + FilterBar + useState dla listy | ~20 min

Cel kroku

Sklej całą aplikację w App.tsx używając useState do przechowywania listy i aktywnego filtru.

Zadanie

Uzupełnij App.tsx zgodnie z poniższym schematem:

- useState<Todo[]> dla listy zadań (inicjalizacja: pusta tablica lub 2–3 przykłady hardcoded),
- useState<FilterType> dla aktywnego filtru (inicjalizacja: 'all'),
- funkcje handleAdd, handleToggle, handleDelete, handleEdit przekazywane jako propsy,
- logika filtrowania: useMemo lub prosta zmienna filteredTodos.

Fragment do analizy – logika filtrowania

```
// src/App.tsx
const [todos, setTodos] = useState<Todo[]>([]);
const [filter, setFilter] = useState<FilterType>('all');

// TODO (3): uzupełnij brakujące przypadki filtru
const filteredTodos = todos.filter(todo => {
  if (filter === 'active') return /* ... */;
  if (filter === 'completed') return /* ... */;
  return true; // 'all'
});
```



```
// Licznik aktywnych zadań
const activeCount = todos.filter(t => !t.completed).length;
```

FilterBar – implementacja

Komponent FilterBar przyjmuje props: activeFilter i onFilterChange. Renderuje 3 przyciski ze stylem aktywnym.

KROK 4

useReducer jako alternatywa dla useState | ~20 min

Cel kroku

Refaktoryzacja zarządzania listą zadań – zamiana useState na useReducer. Jeden slajd koncepcyjny, reszta to kod.

useState	useReducer
Prosty stan (string, number, boolean)	Złożony stan z wieloma polami
Niezależne porcje stanu	Stan z wieloma powiazanymi akcjami
Logika mutacji inline w handlerach	Logika mutacji w izolowanej funkcji (łatwa w testach)
Dobre dla formularzy, toggli, liczników	Dobre dla list CRUD, maszyn stanów

Zadanie – uzupełnij reducer

Plik src/reducers/todoReducer.ts zawiera gotowy szkielet. Uzupełnij oznaczone akcje:

```
// src/reducers/todoReducer.ts
import { Todo, TodoAction } from '../types/todo.types';

export function todoReducer(state: Todo[], action: TodoAction): Todo[] {
  switch (action.type) {

    case 'ADD':
      // TODO (4a): zwróć nową tablicę z nowym Todo na początku
      // Wzór: return [{ id: ..., title: ..., completed: false, createdAt: new
      Date() }, ...state]
      return state;

    case 'TOGGLE':
      // TODO (4b): zmień pole completed dla Todo o id === action.payload
      return state;

    case 'DELETE':
      // TODO (4c): odfiltruj Todo o id === action.payload
      return state;
```

```
    case 'EDIT':
      return state.map(t =>
        t.id === action.payload.id ? { ...t, title: action.payload.title } : t
      );

    default:
      return state;
  }
}
```

Zmiana w App.tsx

Po implementacji reducera zastąp `useState<Todo[]>` na `useReducer`:

```
// PRZED: const [todos, setTodos] = useState<Todo[]>([]);
// PO:    const [todos, dispatch] = useReducer(todoReducer, []);

// Przykład dispatcha w handleAdd:
const handleAdd = (title: string) => {
  dispatch({ type: 'ADD', payload: title });
};
```

KROK 5

Props drilling vs Context API – przykład dydaktyczny | ~15 min

Cel kroku

Zrozumienie problemu props drilling na odrębnym, minimalnym przykładzie dydaktycznym (nie w aplikacji To-Do).

Problem – props drilling

Wyobraź sobie aplikację z drzewem komponentów: `App > Layout > Sidebar > ThemeButton`. Tylko `ThemeButton` potrzebuje wartości `theme`, ale `Layout` i `Sidebar` muszą ją przekazywać mimo że same jej nie używają. To właśnie props drilling.

```
// Problem: theme jest przekazywany przez każdy poziom drzewa
function App() {
  const [theme, setTheme] = useState<'light' | 'dark'>('light');
  return <Layout theme={theme} setTheme={setTheme} />;
}

function Layout({ theme, setTheme }: ...) {
  return <Sidebar theme={theme} setTheme={setTheme} />; // Layout nie używa theme!
}

function Sidebar({ theme, setTheme }: ...) {
```

```
    return <ThemeButton theme={theme} setTheme={setTheme} />; // Sidebar też nie!
  }
function ThemeButton({ theme, setTheme }: ...) {
  return <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
    Zmien motyw
  </button>;
}
```

Rozwiązanie – Context API

Uzupełnij plik `src/context/ToDoContext.tsx` zgodnie z poniższym schematem:

```
// src/context/ToDoContext.tsx (przykład z Theme dla przejrzystości)
import { createContext, useContext, useState, ReactNode } from 'react';

interface ThemeContextType {
  theme: 'light' | 'dark';
  setTheme: (t: 'light' | 'dark') => void;
}

// TODO (5a): utwórz ThemeContext za pomocą createContext
// const ThemeContext = ...

export function ThemeProvider({ children }: { children: ReactNode }) {
  const [theme, setTheme] = useState<'light' | 'dark'>('light');
  // TODO (5b): opakuj children w ThemeContext.Provider przekazując { theme, setTheme }
  return /* ... */;
}

// Custom hook do konsumpcji kontekstu
export function useTheme() {
  // TODO (5c): zwróć wynik useContext(ThemeContext)
  // Wskazówka: sprawdź czy kontekst nie jest undefined
}
```

Efekt po poprawnej implementacji

ThemeButton wywołuje `useTheme()` bezpośrednio i nie wymaga żadnych propsów. Layout i Sidebar są całkowicie przezroczyste dla motywu.

- App opakowuje drzewo w `<ThemeProvider>`
- ThemeButton: `const { theme, setTheme } = useTheme()` – bez żadnych propsów
- Dodanie nowego konsumenta kontekstu nie wymaga modyfikacji pośrednich komponentów

05 Szkielet kodu (skeleton code)

Poziom: lekki – student uzupełnia 1–3 linie w gotowych funkcjach

Poniżej zebrano wszystkie miejsca do uzupełnienia w jednym przeglądzie. Każde TODO jest ponumerowane zgodnie z krokami z Sekcji 4.

TODO	Plik	Zadanie do uzupełnienia	Wzorcowe rozwiązanie (1 linia)
(1)	AddTodoForm.tsx	Deklaracja stanu inputValue	<code>const [inputValue, setInputValue] = useState<string>("");</code>
(2)	AddTodoForm.tsx	Walidacja i wywołanie onAdd	<code>if (inputValue.trim()) { onAdd(inputValue.trim()); setInputValue(""); }</code>
(3)	App.tsx	Brakujące przypadki filtru	<code>active: !todo.completed completed: todo.completed</code>
(4a)	todoReducer.ts	Akcja ADD	<code>[{ id: crypto.randomUUID(), title: action.payload, completed: false, createdAt: new Date() }, ...state]</code>
(4b)	todoReducer.ts	Akcja TOGGLE	<code>state.map(t => t.id === action.payload ? { ...t, completed: !t.completed } : t)</code>
(4c)	todoReducer.ts	Akcja DELETE	<code>state.filter(t => t.id !== action.payload)</code>
(5a)	TodoContext.tsx	Utworzenie kontekstu	<code>const ThemeContext = createContext<ThemeContextType undefined>(undefined);</code>
(5b)	TodoContext.tsx	Opakowanie w Provider	<code>return <ThemeContext.Provider value={{ theme, setTheme }}>{children}</ThemeContext.Provider>;</code>
(5c)	TodoContext.tsx	Custom hook useTheme	<code>const ctx = useContext(ThemeContext); if (!ctx) throw new Error('Brak ThemeProvider'); return ctx;</code>

UWAGA

Wzorcowe rozwiązania są dostępne w galezi main repozytorium – uczestnicy powinni samodzielnie dojść do rozwiązania w trakcie laboratorium. Prowadzący może ujawnić rozwiązania po upływie czasu danego kroku.

06 Rozszerzenia dla zaawansowanych

Dla studentów, którzy ukończyli zadanie podstawowe przed czasem

6.1 Persystencja – custom hook useLocalStorage<T>

Cele: zapoznanie z generycznością w TypeScript, wzorec custom hooka, zapis stanu do localStorage.

```
// src/hooks/useLocalStorage.ts
import { useState, useEffect } from 'react';

export function useLocalStorage<T>(key: string, initialValue: T): [T, (v: T) => void] {
  const [storedValue, setStoredValue] = useState<T>(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? (JSON.parse(item) as T) : initialValue;
    } catch {
      return initialValue;
    }
  });

  const setValue = (value: T) => {
    setStoredValue(value);
    window.localStorage.setItem(key, JSON.stringify(value));
  };

  return [storedValue, setValue];
}

// Użycie w App.tsx:
// const [todos, setTodos] = useLocalStorage<Todo[]>('todos', []);
```

WSKAZOWKA

Hook useLocalStorage można połączyć z useReducer przez opakowanie dispatch: każda akcja serializuje nowy stan do localStorage po zmianie. To wzorec middleware w miniaturze.

6.2 Animacje – Framer Motion

Cele: poznanie biblioteki Framer Motion, animowane dodawanie i usuwanie elementów listy z AnimatePresence.

```
// src/components/ToDoList.tsx (z animacjami)
```

```
import { motion, AnimatePresence } from 'framer-motion';

export function TodoList({ todos, onToggle, onDelete }: TodoListProps) {
  return (
    <ul>
      <AnimatePresence>
        {todos.map(todo => (
          <motion.li
            key={todo.id}
            initial={{ opacity: 0, y: -12 }}
            animate={{ opacity: 1, y: 0 }}
            exit={{ opacity: 0, x: 48 }}
            transition={{ duration: 0.22 }}
          >
            <TodoItem todo={todo} onToggle={onToggle} onDelete={onDelete} />
          </motion.li>
        ))}
      </AnimatePresence>
    </ul>
  );
}
```

Ważne: AnimatePresence musi otaczać listę elementów z kluczem key. Framer Motion śledzi dodawanie i usuwanie po zmianie kluczy.

Powodzenia!